AutoBuild: A Boundary-Aligned Replay Protocol for Repository-Task Validity

Anonymous Author(s)
Affiliation
Address
email

Abstract

In repository-task synthesis pipelines, grading a generated repository in the same mutable environment in which it was developed can make the score depend on undeclared packages, modified runtime state, caches, external files, or background processes rather than on the portable artifact. A fresh fixed grader removes this confound, but source stripping, test sanitization, export, overlay, collection, and parsing can themselves corrupt the task. We call either mismatch *evaluation-path drift*. AutoBuild is a *boundary-aligned replay protocol*: derive task artifacts from a common repository snapshot, establish a known-good pre-transform reference outcome, and send the same reference through candidate packaging and the deployed judge after transformation. The strict target contract compares exact test identities and outcomes, fails closed, exercises negative controls, and requires cross-grader parity. In the inspected Python/pytest snapshot, the source-side check and artifact builders are implemented; post-transform replay is a standalone marker-based procedure, while exact identities and negative controls remain proposed. Source-visible incidents and a qualified retrospective report motivate the failure mechanisms, but missing row-level manifests preclude claims that individual transitions are corrections or that downstream gains are causal. We therefore contribute a problem formulation, an evidence-bounded protocol, and a controlled validation design—not a demonstrated improvement in task validity.

1 Introduction

A whole-repository coding agent changes more than files. While developing a solution, it may install packages or system libraries, alter environment variables and runtime versions, populate caches, create files outside the workspace, or leave services running. If a grader merely restores official tests in that same container, a passing score can reflect the history of the development environment rather than the repository the agent would deliver. This distinction matters for long-horizon benchmarks such as NL2Repo-Bench and document-to-repository evaluation [3, 2]: one execution score becomes both a capability measurement and, in data-generation settings, a label for selecting trajectories.

Hiding tests does not solve this problem. A candidate that silently imports a package installed during development can pass after its tests are overwritten yet fail in the declared target environment. The natural remedy is clean-room grading: export only the candidate artifact and run it inside a fresh fixed image whose runtime, dependencies, evaluator-owned tests, command, and scorer are controlled by the judge. The score then measures whether the delivered repository works under the task contract, rather than whether it can continue to exploit a container it has modified.

Clean-room grading solves one trust problem but creates another. Constructing the grader requires removing the reference source while preserving dependencies and hidden tests. Delivering a candidate requires selecting paths, deleting candidate-authored tests, overlaying files, reinstalling the package,

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

collecting tests, and parsing heterogeneous output. These transformations can change the executable object after the source repository has passed its original check. Over-broad cleanup can delete valid implementation and create a false low; incomplete cleanup can retain reference code or candidate tests and create a false high; a selected-test denominator combined with whole-file collection can assign full reward despite additional failures.

We call either failure *evaluation-path drift*: the verdict depends on undeclared development state or on behavior-changing transformations performed after an earlier validation point. Our key insight is that isolation and transformation validation must be designed together. AutoBuild follows an *Isolate–Transform–Replay* (ITR) design. *Isolate* makes the fixed grader, rather than the agent workspace, the trusted execution base. *Transform* constructs a specification, test interface, cleanup policy, and clean image from shared executable evidence. *Replay* targets the residual construction risk: the same known-good reference is observed before transformation and then sent through candidate packaging and the deployed judge after transformation.

The two runs are not independent semantic oracles. Agreement shows that packaging preserves one known-good implementation, but it cannot show that an empty, stubbed, copied, or adversarial candidate fails. We therefore formulate a strict lifecycle contract with exact test identities, fail-closed process status, negative controls, and parity across delivered graders. Figure 1 shows the complete trust boundary.

Contributions.

- We formulate evaluation-path drift as a two-sided validity problem: mutable development state creates state-conditioned false highs, while the destructive transformations required for clean-room grading create false lows, false highs, and silent test-identity changes.
- We present an evidence-bounded ITR design that combines artifact-only export, shared-snapshot construction, and boundary-aligned replay of one known-good implementation. We explicitly separate integrated, standalone, and proposed components in the inspected snapshot.
- We specify falsifiable evaluation criteria and matched experiments that separate environment isolation, packaging correctness, specification grounding, and downstream training effects. We retain report-only incident counts as motivation rather than treating them as adjudicated corrections.

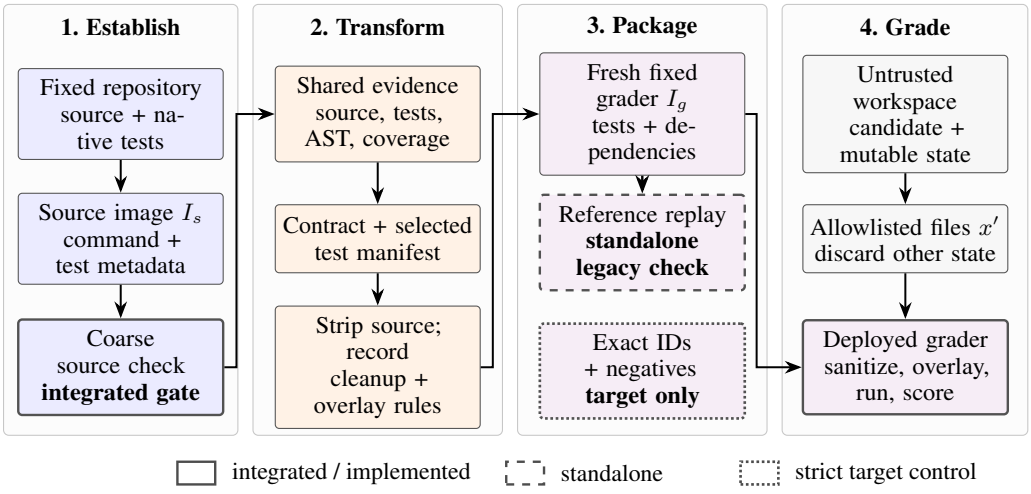

Figure 1: The ITR lifecycle and implementation status. The untrusted workspace contributes only allowlisted files to a fixed grader. The inspected snapshot integrates a coarse source-side check, implements artifact builders, and provides post-transform reference replay as a standalone procedure; strict identity and negative controls are target requirements.

2 Stateful Evaluation-Path Drift

2.1 Artifact determinacy

Let $r = (S, \mathcal{T}, c)$ be a reference implementation, native test suite, and fixed commit. During generation, a trajectory η produces a mutable workspace

$$W_\eta = (x_\eta, \sigma_\eta), \tag{1}$$

where x_η denotes candidate files and σ_η denotes all other accumulated state, including installed software, global configuration, environment variables, external files, caches, and processes. Restoring official tests in place gives

$$J_{\text{reuse}}(W_\eta) = \phi(\text{Run}[W_\eta \oplus \mathcal{T}, C])\,. \tag{2}$$

A task builder instead emits $z = (I_s, I_g, d, C, H, \mathcal{I}, \phi)$: source and grading images, specification, command, export/sanitization policy, intended test-ID set, and scorer. The clean-room judge exports an allowlisted artifact and starts from fixed judge state:

$$J_z(W_\eta) = \phi(\text{Run}[I_g \oplus \text{Export}_H(W_\eta), C])\,. \tag{3}$$

Relative to a frozen dependency and execution contract, we require *artifact determinacy*: if two workspaces export identical candidate artifacts, their authoritative scores are equal. Equation 2 can violate this property because it consumes σ_η; Equation 3 removes that state. If I_g omits a declared legal dependency, the judge violates the frozen contract and the comparison does not diagnose a candidate error.

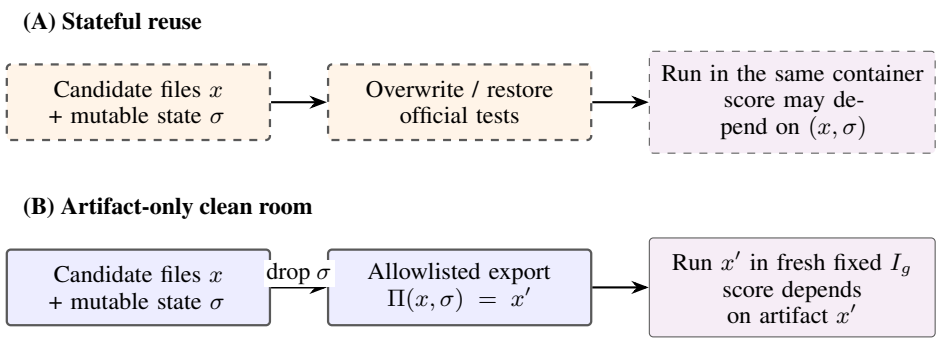

Figure 2: Replacing tests in an agent-modified container does not reset undeclared state. Clean-room grading exports only the candidate artifact into a fresh fixed judge, making the score independent of development history.

2.2 Isolation introduces a transformation boundary

Let $\mathbf{y}_s(S; \mathcal{I})$ and $\mathbf{y}_g(S; \mathcal{I})$ be per-test outcomes for the same reference and intended test identities before and after task packaging. The central non-implication is

$$\text{BuildPass}(S, I_s, C) \implies \text{GradePass}(S, I_g, H, C, \mathcal{I}, \phi). \tag{4}$$

Artifact-path drift occurs when $\mathbf{y}_s \neq \mathbf{y}_g$. Since S is known good, the disagreement localizes failure to source stripping, candidate cleanup, overlay, collection, execution, or parsing rather than to an unknown model solution.

Table 1: Failure mechanisms and the controls needed to expose them. A positive reference replay cannot detect every false high.

Mechanism	Direction	Required diagnostic
Retained development state	false high	same artifact in reused vs fresh environment
Over-broad source/test cleanup	false low	reference through the deployed candidate path
Residual reference or candidate tests	false high	empty/stub/adversarial negative controls
Test-ID/denominator/parser drift	either	exact IDs, outcomes, return code, grader parity

The denominator case illustrates why scalar agreement is weak. If $N = |\mathcal{I}|$ counts selected node IDs but pytest executes their entire files, the scorer

$$\phi(P, F, E; N) = \min(P/N, 1) \tag{5}$$

returns one for $P = N$ even when $F > 0$. Cardinality also misses replacement of one intended test by one unintended test. A lifecycle contract must therefore preserve identities and outcomes, not only a passed-count ratio.

The strict acceptance rule is therefore qualitative rather than prevalence-based: an accepted task must have zero static/runtime identity mismatch, no unclipped $P/N > 1$, and no full score coexisting with a failed or errored test. Natural incidence and parametrization stress remain secondary diagnostics in Appendix D.

3 AutoBuild: Protocol and Audited Snapshot

ITR gives each stage one responsibility: the fixed image establishes the grading boundary; shared executable evidence grounds the task artifacts; target replay compares a known-good reference on both sides of the destructive transform. The checked code contains narrower legacy checks at those locations. Table 2 prevents the target contract from being mistaken for a current batch invariant.

Table 2: Status in the inspected Python/pytest snapshot.

Component	Status	Evidence-bounded guarantee
Source-side A_1^{legacy}	integrated	coarse buildability gate
Spec./clean-image builders	implemented stages	shared snapshot and tests; bridge absent
Candidate isolation	one evaluator path	fresh fixed grader receives candidate artifacts
Post-transform A_2^{legacy}	standalone	marker-based positive check; not a batch gate
Exact boundary replays, negatives, parity	proposed	strict target contract; not yet evaluated

3.1 Reference image and integrated legacy check

Given a fixed repository commit, language-model roles locate dependency and test configuration, install and build the project, exercise native tests, and organize rebuild commands, test commands, and per-test status metadata. The resulting source image I_s anchors the later task artifacts. The integrated legacy check reloads I_s, executes the extracted commands, defines $T_s = P_s + F_s + E_s$, and applies

$$A_1^{\text{legacy}}(z; \tau) = \mathbf{1}[T_s > 0 \ \wedge \ P_s/T_s \geq \tau], \qquad \tau = 0.9 \text{ by default.} \tag{6}$$

Failure stops the instance early. This is a triage gate, not an exact oracle: the implementation parses a coarse stdout regular expression, ignores process return code, omits skipped tests, and does not equate its parsed tests with the richer per-test identities selected downstream.

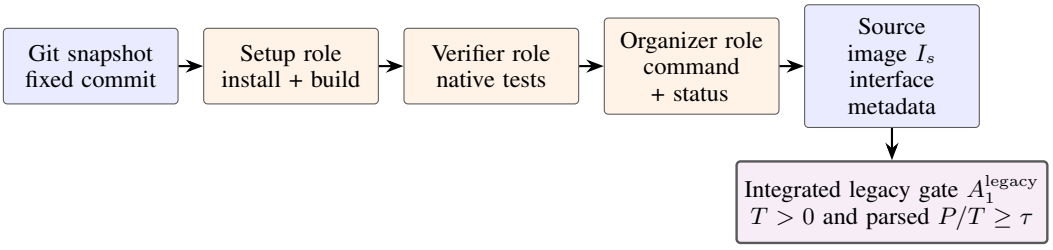

Figure 3: Source-image construction and the integrated legacy source check. It asks whether a known-good starting point remains sufficiently runnable after environment construction; it does not satisfy the exact target replay or exercise candidate delivery.

3.2 Shared-evidence specification synthesis

A tool-using documentation agent reads source and tests, retrieves symbol bodies, and queries runtime signatures. Before generation, AutoBuild extracts an AST inventory, profiles execution coverage in I_s, and identifies names referenced by tests. Public and tested-private symbols form a deterministic MUST-COVER set; when the public top-level surface exceeds 50 symbols, a bounded tested-first set is used.

The agent writes a from-scratch, installable contract. A bounded repair loop checks that each required symbol name appears in a `def` or `class` signature and detects fenced blocks containing at least four consecutive normalized lines copied from source or tests. At most three revisions are attempted. This is a *coverage-guided contract gate and copied-code detector*, not a semantic proof: the default check does not compare parameters or defaults, the stronger critic is disabled, unresolved items can become `TODO` contracts, and paraphrased test logic can escape detection. Direct access to evaluator tests also motivates a held-out behavioral split unavailable to the generator.

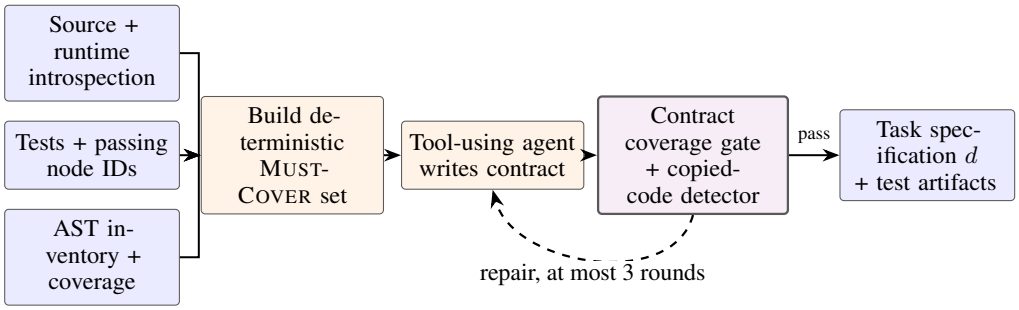

Figure 4: Test-grounded task synthesis in the inspected snapshot. Source, tests, AST structure, and coverage share one evidence base; the deterministic gates provide bounded structural grounding rather than semantic completeness or leakage freedom.

3.3 Clean grading image and standalone legacy replay

The clean-image builder relocates the workspace, preserves test-shaped files and test directories, removes reference implementation files, and cleans source-control metadata, installed-package copies, wheel caches, bytecode, and build remnants. It rejects an empty import surface and injects static version metadata for packages that derive versions from Git. During delivery, one deployed evaluator removes explicit test paths and pytest-discoverable `test_*.py`, `*_test.py`, and `conftest.py` files before candidate overlay. Other grader paths in the snapshot do not apply identical cleanup, so parity is not established.

The standalone post-transform script packages S as if it were a candidate: it removes listed tests and packaging files, overlays the remainder into I_g, installs the package, runs the delivered command, and computes

$$Q_2^{\text{legacy}}(z) = \min(P_g/N, 1), \qquad A_2^{\text{legacy}}(z) = \mathbf{1}[N > 0 \wedge P_g = N]. \tag{7}$$

The operating procedure requires score one and an `ORACLE_PASS` marker. However, the batch clean-image entry point does not invoke the script, and an unsuccessful marker normally still exits with status zero. We therefore treat A_2^{legacy} as a documented spot-check in the inspected snapshot, not proof that every task passed both legacy checks.

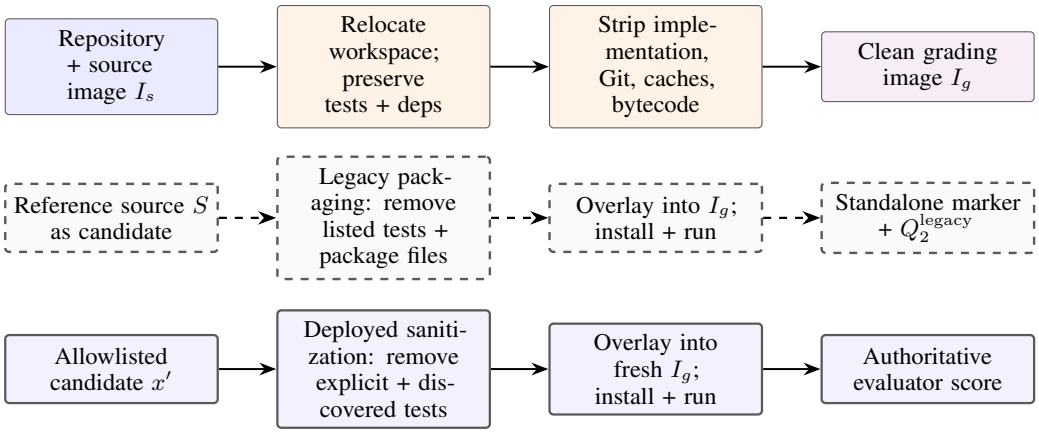

Figure 5: Clean-image construction, standalone legacy replay, and the deployed evaluator path. The standalone script uses candidate-like packaging but differs from the deployed evaluator's sanitization, so cross-path parity is not established; it is also not batch-enforced.

3.4 Strict acceptance contract

The legacy scalars in Equations 6 and 7 use different parsers and acceptance semantics; equality between them is therefore not a meaningful strong invariant. The target design instead freezes a manifest $\mathcal{I}_m$ and executes an exact source replay R_s and an exact post-transform replay R_g with the same command semantics. It accepts only if

$$\mathcal{I}_s^c = \mathcal{I}_s^p = \mathcal{I}_g^c = \mathcal{I}_g^p = \mathcal{I}_m, \qquad F_s = E_s = K_s = F_g = E_g = K_g = 0, \qquad \mathrm{rc}_s = \mathrm{rc}_g = 0, \quad (8)$$

where superscripts c, p denote collected and passed IDs and K denotes skips under a declared policy. This exact target realizes the intended "scores agree" rule through equality of test identities and outcome vectors rather than legacy scalar equality. Installation, collection, execution, parsing, or identity failure must return nonzero. Positive replay is paired with empty/import-only candidates, behavior stubs, source mutants, and adversarial workspaces containing candidate tests, hooks, packaging overrides, or source-retrieval attempts. Finally, the same candidate and manifest must produce identical outcome vectors in every delivered grader. None of these strict additions is batch-enforced in the inspected snapshot.

4 Inspection Evidence and Motivating Records

We distinguish implementation-visible behavior, counts embedded in source comments or operating notes, and an internal retrospective report. The latter two lack row-level manifests in the checked workspace and are not independently reproduced measurements.

The source-stripping repair contains a comment attributing 86 of 470 all-zero tasks in one cohort to deleting non-standard tests colocated with implementation. A distinct cleanup-path comment lists 103 affected IDs after a test directory, rather than the exact test file, was used to sanitize candidates and deleted in-package implementation. A documentation-generation comment records 24 of 380 rows with hollow or duplicate canonical sections before a structural repair. The operating procedure records one end-to-end smoke instance with 31 selected tests passing in the source image and score one after clean packaging. These records connect the constructive failures to engineering incidents, but they do not estimate prevalence beyond their separate cohorts.

An internal retrospective report also records substantial score transitions after grading-path repairs. Because transition-level logs and independent adjudication are unavailable, we treat it only as scale motivation: it cannot identify which repair caused a change or whether any individual transition moved toward ground truth. Appendix A preserves the provenance-qualified counts.

A correction claim additionally requires immutable row-level lineage, cohort reconciliation, and blinded adjudication showing a net label-error reduction with a confidence interval excluding zero.

Table 3: Provenance-qualified internal report. These counts motivate scale only; they are not measured effectiveness results.

Quantity	Count	Qualifier
Rollouts re-evaluated	35,503	reported exact count
Scores changed	$\approx$10,000	rounded report
Zero $\rightarrow$ positive	$>$4,000	about 11.3% lower bound

No raw configuration or evaluation log supporting the preliminary student-model, teacher-score, token-budget, learning-rate, or Doc2Repo values in the planning memo was found in the checked project, source history, local paper materials, or nearby experiment-like files. We consequently exclude those values from the paper's visible empirical claims.

5 Evaluation Protocol

The artifact audit in Section 4 establishes which mechanisms are present in the inspected snapshot. We use the following controlled protocol to evaluate the effectiveness of the proposed exact replay policy while keeping construction quality, evaluation state, and downstream training effects distinct.

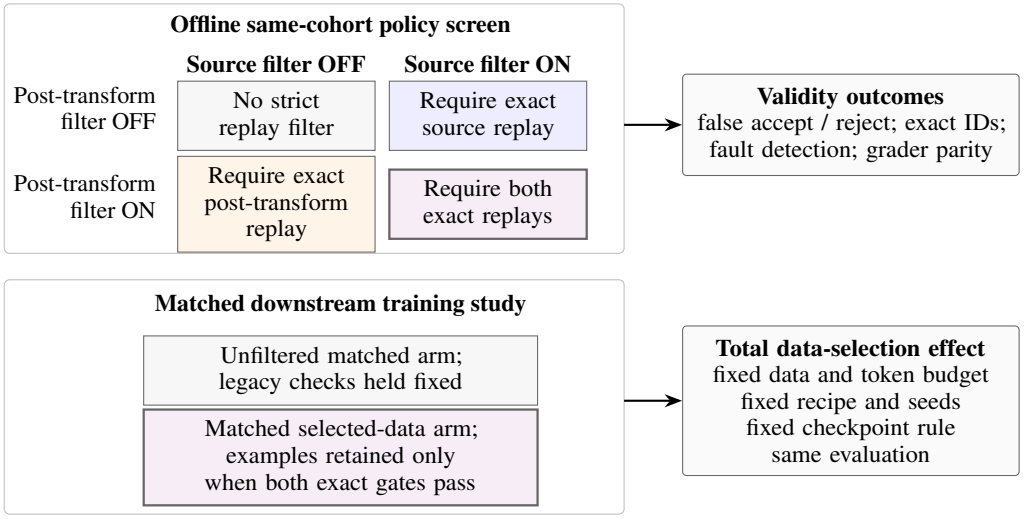

Figure 6: Validation design. The 2×2 settings are policy filters on one offline cohort, not per-task pass labels. A separate matched two-arm training study estimates only the total effect of requiring both exact gates under fixed data volume and recipe.

Frozen matched manifest. Freeze one construction-complete master pool, report each policy's eligible pool, and subsample equal-sized training arms with equal optimizer updates and supervised tokens (at most a predeclared 0.5% packing-rounding tolerance). Use at least three seeds, fixed final-checkpoint selection, complete lineage, and one pinned benchmark/evaluator digest.

Environment isolation. Evaluate each exported candidate with identical official tests and scoring in (A) its agent-modified work container and (B) a fresh fixed grader. Inject controlled dependencies on undeclared packages, system libraries, runtime versions, environment variables, caches, external files, and processes. Report the paired score-transition matrix, artifact-determinacy violations, A-pass/B-fail rate, and adjudicated failure causes. Fix the declared dependency contract before evaluation.

Boundary-replay ablation. Apply a factorial $R_s \in \{0,1\} \times R_g \in \{0,1\}$ policy screen to one construction-complete cohort: no strict replay, exact source replay only, exact post-transform replay only, and both. Log the integrated coarse source check and standalone marker check separately as legacy baselines. Inject faults into source stripping, candidate cleanup, overlay precedence, test

collection, execution, and parsing. Separately report rejection and retention of valid tasks, escape of invalid controls, reference and legal-alternative pass, empty/stub/mutant/adversarial outcomes, exact-ID violations, grader disagreement, time, and cost. If one gate subsumes the other, do not claim synergy.

Downstream total data effect. Compare a legacy-baseline arm and a full-strict-policy arm drawn from the same master pool. Both arms satisfy identical upstream feasibility requirements and differ only in whether exact R_s/R_g, negative-control, and parity policies select examples. This estimates only the *total* selection effect; identifying separate contributions from R_s and R_g requires four matched training arms. Match task count and token budget, and hold teacher setting, student model, optimizer, schedule, training budget, checkpoint selection, seeds, and evaluation fixed. Report NL2Repo-Bench mean test pass rate and Pass@1 separately from BeyondSWE-Doc2Repo mean test pass rate and full/$\geq 90\%$ completion counts [3, 2].

6 Related Work

Whole-repository generation. NL2Repo-Bench gives an agent one requirements document and an empty workspace, then evaluates the packaged Python library in a separate test image using the upstream pytest suite; it reports mean test pass rate and Pass@1 [3]. Concurrent BeyondSWE evaluates four task families; *Doc2Repo* is its 50-instance document-to-repository subset rather than a separate paper [2]. It reports mean pass rate and the numbers of fully correct and at-least-90%-correct repositories. BeyondSWE already extracts a generated repository from the agent workspace, applies it in a fresh container, restores evaluator tests, and scores it. We therefore do not claim fresh-container judging, hidden-test restoration, or document-to-repository evaluation as new.

Executable task construction. SWE-smith constructs working Python environments before synthesizing faults that break previously passing tests [7]. Concurrent ScaleSWE coordinates environment, test, and test-grounded problem-statement agents and retains tasks with the required buggy/patched transitions [8]. RepoLaunch produces reproducible builds, per-test commands, and structured status parsers across repositories and platforms [4]. Closest to our data setting, concurrent DeNovoSWE profiles repositories with tests and coverage, synthesizes documentation with critic–repair loops, strips source and recovery channels, filters trajectories, and evaluates downstream transfer to NL2Repo-Bench and BeyondSWE-Doc2Repo [9]. Environment-first construction, test-grounded specifications, source stripping, and score-based filtering are consequently prior or concurrent ingredients.

Audit and lifecycle validation. SWE-Bench+ audits solution leakage and weak tests in repository-repair tasks [1]. Concurrent Auto Benchmark Audit scales agentic detection of hidden environment dependencies, specification gaps, and brittle graders, then studies ranking changes after expert-validated filtering [6]. Concurrent Change2Task validates healthy, constructed-task, and restored repository states when transferring historical changes to modern revisions [5]. Thus, neither benchmark auditing nor multi-stage executable validation is by itself new.

Table 4: Positioning against the closest systems. "Not reported" means not stated as a primary guarantee in the cited paper. The last row exposes the gap between the inspected snapshot and our target contract.

System	Fresh grader	Transform	Reference lifecycle	IDs / invalid controls
BeyondSWE	yes	extract submission; restore tests	not reported as a construction gate	not reported
DeNovoSWE	yes	strip source and tests	source-side pass/coverage filter	no strict contract reported
Change2Task	task environments	reconstruct history into a task	healthy–task–restored validation	regression, scope, and fidelity checks
AutoBuild snapshot	evaluator path	strip source; sanitize candidate	coarse source check integrated; post replay standalone	exact IDs and negatives proposed

Scope. Our boundary is narrower: isolation requires destructive transformations, so source stripping, candidate export, test removal, overlay, collection, and score aggregation can make the deployed judge disagree with the source-admission path. AutoBuild replays a known-good reference after

this transform through the same candidate and grading path, and combines the result with test-identity, process-status, denominator, negative-control, and parity requirements. This complements BeyondSWE's fresh-container evaluation, DeNovoSWE's synthesis pipeline, ABA's task-level audit, and Change2Task's maintenance-task lifecycle. Any downstream training claim still requires matched filtering and training experiments.

7 Limitations and Broader Impacts

Evidence and scope. The inspected snapshot is specialized to Python/pytest. Its second replay is not batch-integrated, grader implementations are not behaviorally identical, and raw row-level transition logs, downstream training records, compute records, and repository-level license manifests are unavailable. The paper therefore supports mechanism and protocol claims, not prevalence, corrected-label, or model-improvement claims.

Guarantees. Reference replay is a positive control, not a semantic oracle. It does not prove that a specification is complete, evaluator tests are adequate, a task is uncontaminated, or an invalid candidate fails. Generator and grader can share test, parser, or dependency mistakes; candidate exports can contain new malicious behavior; and a fresh judge can still omit a legal dependency. The current copied-code detector and contract gate are deliberately bounded. Network, process, filesystem, and secret isolation, held-out tests, negative controls, and differential graders remain necessary.

Broader impacts. More reliable labels can reduce wasted training compute and misleading capability claims. Conversely, harvesting public repositories raises license, attribution, privacy, supply-chain, and redistribution concerns; test-grounded specifications may expose behavior authors did not intend to publish. A responsible release should preserve repository provenance and licenses, scan secrets, honor removal requests, document task-specific limitations, and red-team both source leakage and false rejection before distributing images or trajectories.

8 Conclusion

Repository-task validity depends on where a candidate is graded and what happens while moving it there. Clean-room grading removes mutable development state, while the proposed boundary-aligned replay protocol tests whether the required isolation transform preserves a known-good task. Exact test identity, fail-closed execution, negative controls, and grader parity define the target contract. The principle is simple: a score should be determined by the exported artifact under fixed judge state, and every destructive transformation should be validated after it becomes consequential.

References

[1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. URL https://arxiv.org/abs/2410.06992.

[2] Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong Wen. BeyondSWE: Can current code agent survive beyond single-repo bug fixing?, 2026. URL https://arxiv.org/abs/2603.03194.

[3] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2025. URL https://arxiv.org/abs/2512.12730.

[4] Kenan Li, Rongzhi Li, Linghao Zhang, Qirui Jin, Liao Zhu, Xiaosong Huang, Geng Zhang, Yikai Zhang, Shilin He, Chengxing Xie, Xin Zhang, Zijian Jin, Bowen Li, Chaoyun Zhang, Yu Kang, Yufan Huang, Elsie Nallipogu, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. RepoLaunch: Automating build and management of code repositories across languages and platforms, 2026. URL https://arxiv.org/abs/2603.05026.

[5] Haomin Qi, Xingliang Wang, Xuanqi Gao, Baihui Sang, Xin Zhang, Minghua Ma, Pengfei Gao, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Change2Task: From repository changes to executable coding agent tasks and environments, 2026. URL https://arxiv.org/abs/2607.28591.

[6] Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. Automated benchmark auditing for AI agents and large language models, 2026. URL https://arxiv.org/abs/2605.26079.

[7] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/8b86cf5ace600c48fd188efbb8dedec8-Abstract-Datasets_and_Benchmarks_Track.html.

[8] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, Peng Wang, Cheng Chen, Ji-Rong Wen, and Kai Jia. Immersion in the GitHub universe: Scaling coding agents to mastery, 2026. URL https://arxiv.org/abs/2602.09892.

[9] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Wayne Xin Zhao, Ruihua Song, Ji-Rong Wen, and Kai Jia. DeNovoSWE: Scaling long-horizon environments for generating entire repositories from scratch, 2026. URL https://arxiv.org/abs/2606.10728.

A Qualified Retrospective Record

After grading-path repairs, an internal report states that 35,503 rollouts were re-evaluated, nearly 10,000 scores changed, and more than 4,000 moved from zero to a positive value. The latter counts are respectively rounded and a strict lower bound. Without transition-level logs or independent adjudication, we call them *evaluator transitions*, not false-zero corrections. They cannot isolate which repair caused a change, establish the direction of ground-truth error, or support confidence intervals.

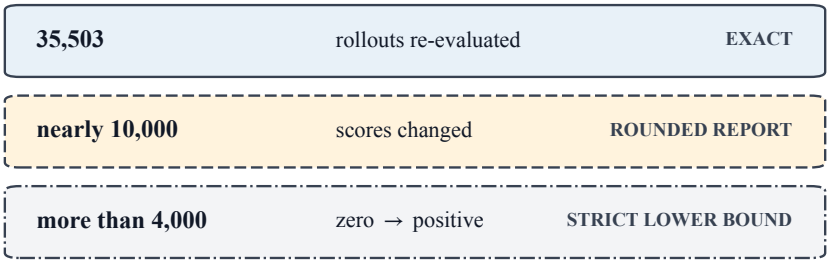

Figure 7: Qualified retrospective incident record. Wording, line style, and badges distinguish the exact cohort size from a rounded report and a strict lower bound. Row-level transitions and adjudicated labels are unavailable.

B Claims-to-Evidence Ledger

Claim	Current evidence	Excluded extrapolation
Legacy source check is an integrated gate	build runner blocks completion below a parsed 0.9 threshold	exact source replay, native-suite equivalence, or parser correctness
Task artifacts share executable evidence	source/test tools, AST inventory, coverage, passing node IDs	bidirectional joint optimization
Legacy post-transform check can exercise a known-good solution	standalone script and one documented smoke task	all production tasks were gated or exact identities were preserved
Cleanup defects occurred	code-embedded 86/470 and 103-ID audit notes	prevalence outside those distinct cohorts
Scores changed after evaluator repairs	report over 35,503 rollouts	every change is a ground-truth correction
Candidate-test defenses exist	explicit and catch-all removal in one evaluator	equal defenses across all grader implementations

C Target Replay Procedure

```
for each repository snapshot r:
    I_src, C, parser, per_test = build_and_organize(r)
    legacy_A1 = execute_coarse(I_src, C)
    require legacy_A1.total > 0
    require legacy_A1.passed / legacy_A1.total >= tau

    ids = freeze(per_test.passing_ids)
    require ids is nonempty
    R_src = execute_exact(I_src, C, ids)        # target
    require R_src.return_code == 0
    require R_src.collected_ids == ids
    require R_src.passed_ids == ids
    require R_src.failed == R_src.errors == R_src.skipped == 0

```

```
    doc = generate_spec(source=r.source, tests=r.tests,
                        ast=inventory(r), coverage=profile(r))
    I_clean, cleanup, manifest = package_clean_task(r, doc)

    R_grade = deployed_judge(candidate=r.source, # target
                             image=I_clean,
                             cleanup=cleanup,
                             tests=manifest)
    require R_grade.return_code == 0
    require R_grade.collected_ids == ids
    require R_grade.passed_ids == ids
    require R_grade.failed == R_grade.errors == 0
    require R_grade.skipped == 0

    for negative in [empty, import_stub, behavior_stub,
                     mutants, adversarial_tests, source_fetch]:
        require deployed_judge(negative).score < acceptance

    require differential_parity(all_supported_graders)
    persist immutable inputs, image digests, commands, outputs,
            parser hash, per-test outcomes, and all gate decisions
```

D Evaluation Details

The primary controlled comparisons are specified in Section 5. The following secondary analyses diagnose why a task is accepted or rejected and prevent aggregate improvements from masking evaluator faults.

Diagnostic	Required output
Specification grounding	Held-out behavior, interface coverage, copied/paraphrased leakage
Learning dynamics	Fixed token checkpoints, per-seed curves, final-checkpoint interval
Identity and fault injection	Exact IDs, denominator failures, per-family strict-union recall
Retention, cost, decontamination	Cohort funnel, runtime cost, frozen overlap report
Adjudication and grader parity	Blinded weighted error change and per-instance cross-grader agreement
Sensitivity, strata, retrospective audit	Threshold stress, scoped subgroup effects, row-linked transition matrix

Table 5: Secondary analyses and their required outputs.

E Implementation Gaps Found by the Audit

The inspected snapshot is a prototype rather than a fully reproducible release. The legacy source-to-clean-image bridge named in the operating procedure is absent. Optional metadata modules associated with difficulty and task-side guards are also absent and import failures are silently ignored. The clean-image batch path does not invoke the legacy post-transform check, the replay marker does not determine process status, and different grading implementations apply different candidate-test, import-source, and post-hoc retrieval defenses. These gaps are why the main paper distinguishes an integrated coarse source check, a standalone marker-based post-transform check, and the proposed exact replay protocol.

F LLM Usage in the Core Method

Language models are core components of environment setup, test-interface organization, and specification generation. The documentation agent uses repository browsing, test reading, symbol-source retrieval, and runtime-introspection tools. The checked snapshot configures the model through external environment variables and does not preserve a complete model-version, prompt-hash, temperature,

or seed record for the reported audit cohorts. A reproducible release must log these values per artifact; writing assistance for this manuscript is separate from the scientific pipeline.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract, Section 1, and Section 4 distinguish implementation facts, report-derived transitions, and evaluation requirements; Section 7 bounds generalization.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 discusses missing raw evidence, language scope, shared-failure modes, incomplete automation, leakage, and release constraints.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper introduces definitions and target invariants, but makes no theorem or proof claim.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Section 4 and Appendix B disclose the available cohorts and mechanisms, but the row-level transition logs needed to reproduce the main retrospective counts are unavailable in this draft.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The audited implementation and internal rollout data are not presently cleared for anonymous public release; Section 7 states this restriction.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [No]

   Justification: We specify the audit materials and known cohort sizes, but exact repository composition, model configuration, and downstream training settings are absent. We therefore exclude causal training claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Exact per-row observations are unavailable, so confidence intervals or error bars cannot be computed. The incident-record table preserves rounded and lower-bound qualifiers without converting them to precise rate estimates.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The incident reports do not preserve complete compute type, runtime, storage, or total-cost records for the audit cohorts.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This is a software-system audit with no human-subject experiments. The draft avoids exposing private repository locations or credentials and discusses responsible artifact release in Section 7.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 7 discusses both reduced label/compute waste and risks involving license compliance, privacy, supply chains, leakage, and false rejection.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This draft does not release a model or dataset. Proposed release safeguards are listed in Section 7.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Prior work is cited, but a repository-by-repository license and terms-of-use audit is not available for the internal task cohorts; no cohort assets are released with this draft.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper analyzes an internal prototype and introduces no publicly released dataset, model, or code asset in this submission draft.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or human-subject research and therefore requires no IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Appendix F describes the language-model roles in environment and specification generation and identifies the missing configuration records needed for reproduction.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.